

FINAL PROJECT PROPOSAL

<Budget Buddy>

2025.10.28

**Prepared for
COM S 3190 - Construction of User Interfaces
Iowa State University Computer Science Department**

Prepared by

**Jongwoo Kim
David Lawlor**

Contents

Contents	2
1. Introduction	3
2. Purpose	3
3. Goals and Objectives.....	4
Objectives.....	4
4. Project Description	5
4.1. Dashboard:.....	5
4.2. How to Budget:.....	6
4.3. Smart Spending:	6
4.4. How to Invest:.....	6
4.5. Authors Page:.....	6
5. Exercising Project Options	7
6. Feature Ownership & Responsibility	8
7. Resources and Tools.....	9
8. Data Sources and Management The project relies on structured JSON files to populate most page content.....	9
9. AI Features (Optional)	10
10. User Experience Views	10
11. Final Comments	12

1. Introduction

We, Jongwoo Kim and David Lawlor, are from team NM_15, majoring Computer Engineering and the other in Computer Science.

David has previous experience with COMS3090, where he worked on building a social media app with a functioning front end and back end in Java. Jongwoo, while new to COM S 3190, brings strong technical and problem-solving skills from his engineering coursework and programming background.

By combining our unique perspectives—David’s frontend experience and Jongwoo’s structured engineering approach—we aim to collaborate effectively to create a practical and user-friendly web application. Our project, Budget Buddy, focuses on applying the principles learned in COM S 3190 to build a meaningful, real-world financial tool.

2. Purpose

Budget Buddy is a personal finance tracking web application designed to help users manage their income, expenses, and savings through a simple and interactive interface.

The purpose of this project is to extend the midterm version of Budget Buddy—originally a static JSON-driven website—into a dynamic and data-driven web app with improved functionality and user interactivity. The application will allow users to:

- Record and categorize income and expenses easily.
- Visualize financial data using pie charts and bar graphs.
- Set monthly budget goals and track remaining funds.
- Receive personalized “smart spending” insights and daily saving challenges.

This project combines creativity and technical depth, enabling us to strengthen our full-stack development skills while applying key concepts from COM S 3190, such as responsive design, modular coding, and data-driven UI generation.

The relevance of Budget Buddy lies in its real-world practicality—students and young professionals can use it to better understand their spending habits, while we gain hands-on experience with frontend and backend integration using React, Node.js, and MongoDB.

3. Goals and Objectives

Goals

- Build a complete full-stack web application using React, Node.js, and MongoDB.
- Design a responsive and accessible interface with consistent navigation.
- Implement data visualization for budgeting insights (pie and bar charts).
- Integrate personalized financial recommendations for enhanced user engagement.

Objectives

- **Week 1 (Nov 10 – Nov 16):**

Set up project structure, initialize Git repository, create wireframes for the main pages (Dashboard, Smart Spending, Authors), and design basic UI components.

- **Week 2 (Nov 17 – Nov 23):**

Build core React UI components and implement JSON-based dynamic content rendering. Connect the frontend to mock backend routes.

- **Week 3 (Nov 24 – Nov 30):**

Implement backend APIs using Node.js and Express. Design MongoDB schema for user transactions and smart spending data.

- **Week 4 (Dec 1 – Dec 7):**

Integrate backend with frontend, implement authentication features, and add visualizations using Chart.js or D3.js.

- **Week 5 (Dec 8 – Dec 14):**

Finalize UI design, conduct testing and debugging, ensure responsive performance, and prepare for final demo presentation

4. Project Description

Budget Buddy is an interactive and data-driven web application that helps users manage their personal finances efficiently.

The main goal of the project is to allow users to record income and expenses, visualize their spending habits, and receive helpful insights for smarter financial planning.

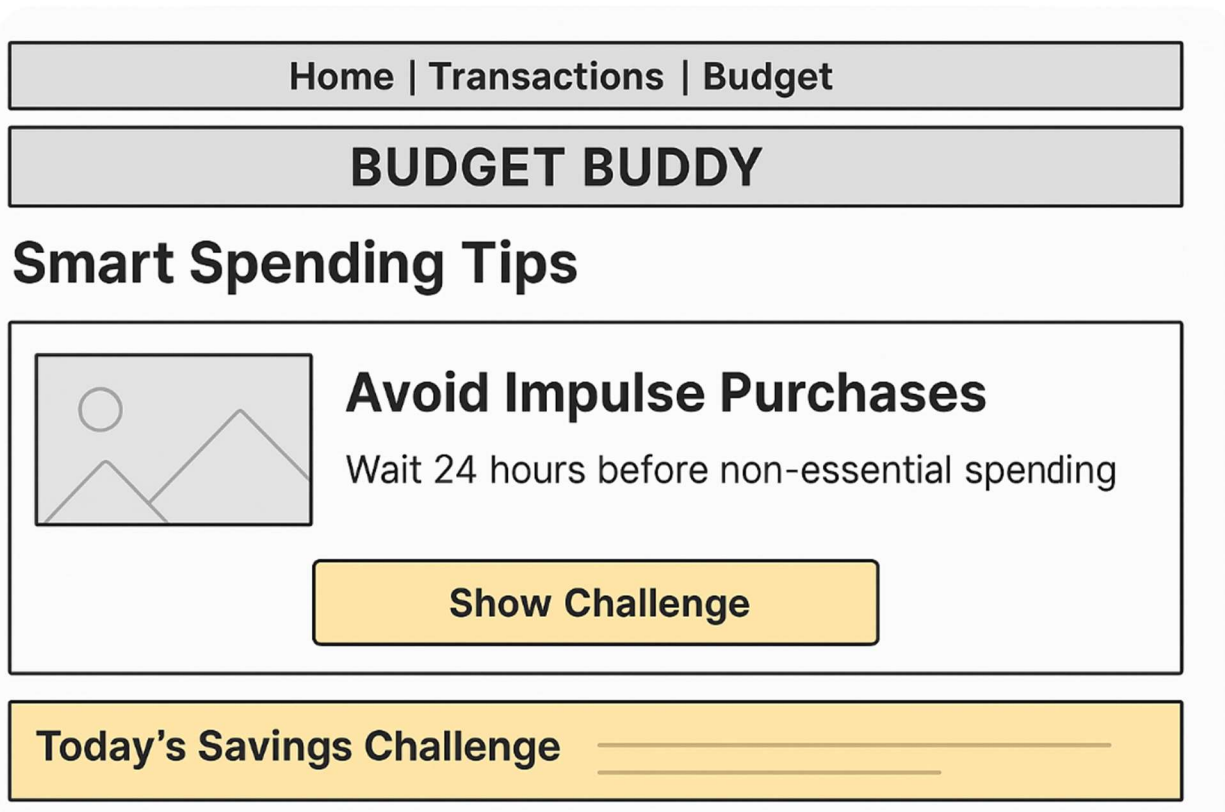
The website consists of five core pages, each serving a unique function:

4.1. Dashboard:

Serves as the landing page and provides an overview of the main features.

Each feature (Budgeting, Smart Spending, Investment, FAQ, and Authors) is represented by dynamic cards loaded from a JSON file.

Clicking a card automatically navigates the user to the corresponding section.



4.2. Record & Categorize Income and Expenses

Allows users to quickly add income and expense items and automatically categorize them.

JavaScript form handlers validate entries (amount, date, category) then normalize and store each record in localStorage (or send to a backend API). A categorization function assigns a category tag (e.g., Housing, Food, Transport, Entertainment, Savings) either by user selection or by simple keyword matching rules. A transaction list is generated dynamically as a Bootstrap table with inline edit/delete controls and filters (by month, category, positive/negative). This makes it painless for users to keep an accurate, searchable ledger without manual spreadsheets.

4.3. Set Monthly Budget Goals & Track Remaining Funds

Lets users create per-category monthly budget targets and then see real-time remaining funds.

A budget builder UI accepts either fixed dollar goals or recommended percentage-based goals. JavaScript uses stored transactions to subtract spending from each category's monthly goal and displays: goal, spent, remaining, and percent-used. Provide progress bars (Bootstrap progress) and warnings when a category hits 75%/90% thresholds. A "Reset month" helper rolls budgets forward while saving historical budget performance.

4.4. Personalized Smart Spending Insights & Daily Saving Challenges

Delivers tailored spending tips and daily micro-challenges to help users save more and spend smarter.

The app analyzes recent spending patterns and produces personalized insights (e.g., "You spent 18% more on dining out this month vs last — consider cooking twice a week"). Insights are generated by JS analysis functions that look for anomalies, rising trends, and opportunities to reallocate funds. The "Today's Savings Challenge" button randomly selects a small, achievable challenge (e.g., "No coffee out today — save \$3", "Pack lunch twice this week — estimated \$12 saved") influenced by the user's categories. Challenges and insights are displayed as dynamic cards; users can mark challenges as completed and track streaks.

4.5. Authors Page:

Displays author information dynamically using authors.json, which contains names, majors, bios, and images.

The JavaScript script loads this JSON data and generates Bootstrap cards for each team member.

This demonstrates modular data handling that can easily scale to include new contributors in the future.

From a technical perspective, Budget Buddy implements the following technologies:

- HTML, CSS, and JavaScript for structure, styling, and interactivity.
- Bootstrap 5 for responsive and consistent UI design.
- JSON files (data.json and authors.json) for structured data management.
- Fetch API and DOM manipulation for dynamic content rendering.
- Chart.js (planned) for visualizing budget and spending distribution in the final version.

The entire project follows a modular and reusable code structure, separating data, logic, and presentation layers.

This ensures maintainability and scalability, allowing the future addition of a backend system for user accounts, authentication, and persistent data storage.

5. Exercising Project Options

Option 1: Extending Midterm Project

For our final project, we are extending the **Budget Buddy** midterm project to include additional features, stronger accessibility, and better performance.

Our key enhancement checklist includes:

- **JSON Schema Expansion:**

Unify and extend JSON structures across *homeTopics*, *smartSpendingTips*, and *investmentTips* sections for consistent field naming.

- **Reusable Card Component:**

Convert repeated card structures (image, title, description, link) into a modular JavaScript generator function.

- **Accessibility Improvements:**

Add descriptive aria-labels, improve keyboard focus navigation, and ensure WCAG-compliant color contrast.

- **Testing and Error Handling:**

Implement light DOM rendering tests and robust error handling for failed fetch() operations.

- **Interactive UI Elements:**

Added buttons, progress bars, and charts that respond to user input, replacing the static text and images.

6. Feature Ownership & Responsibility

Each team member will take full end-to-end ownership of specific features, ensuring balanced workload and accountability.

- **Jongwoo Kim:**

Dashboard, Smart Spending (including “Today’s Challenge” logic), Authors Page, unified navigation styling, and front-end design consistency.

- **David Lawlor:**

Budgeting Page (including Budget Builder and Chart Visualization), Investment Page (JSON-driven expandable sections), and FAQ Page (Bootstrap accordion and dynamic content integration).

Both members will collaborate on JSON structure design, accessibility validation, and GitLab issue tracking.

7. Resources and Tools

Frontend:

HTML5, CSS3, Bootstrap 5, Vanilla JavaScript

Data Management:

JSON (data.json, authors.json), optional JSON Schema validation

MongoDB

Collaboration:

GitLab (issues, branches, merge requests), Discord (communication & code reviews)

Testing & Debugging:

Browser DevTools, basic DOM render verification, optional Playwright smoke tests

Deployment:

Local Live Server or VSCode Live Share for consistent testing environment

8. Data Sources

Primary Data Source: All application data for Budget Buddy will be stored and managed using MongoDB Atlas, a cloud-based NoSQL database. Its flexible document model is ideal for handling diverse data types such as transactions, budgets, user profiles, and spending insights without requiring rigid schemas. Collections like users, transactions, and budgets will allow fast queries and aggregation for charts and reports. MongoDB Atlas also provides built-in security, automated backups, and scalability for growing user demand. Overall, it offers the flexibility and performance needed to support Budget Buddy's evolving financial features efficiently.

Flow:

fetch(data.json) → Check existence of each section → Dynamically render cards via DOM →

If missing or invalid data, display user-friendly warning message (using Bootstrap alert).

Integrity Rules:

- Required fields: id, title, description

- Optional fields: image, link, alt
- Validation: Ensure all cards have unique IDs and accessible alt text

9. AI Features

We plan to introduce a lightweight optional enhancement using basic rule-based logic.

- **Challenge Recommendation Logic:**

Improve “Today’s Savings Challenge” feature to consider user interaction history and categorize challenges by difficulty.

- **Auto Summary Feature:**

Generate short tip summaries dynamically using keyword extraction, instead of static descriptions.

These features will remain optional and primarily demonstrate the potential integration of simple AI-assisted logic into the project.

10. User Experience Views

The user interface is divided into distinct pages that follow a consistent design and navigation flow.

- **Dashboard:** Entry point of the website, displaying feature overview cards loaded dynamically from JSON.
- **How to Budget:** Users input their monthly budget → App calculates and visualizes allocation using bar and pie charts.

- **Smart Spending:** Shows dynamically generated saving tips → Button click triggers random “Today’s Challenge” display with fade-in animation.

Home | Transactions | Budget

BUDGET BUDDY

Smart Spending Tips

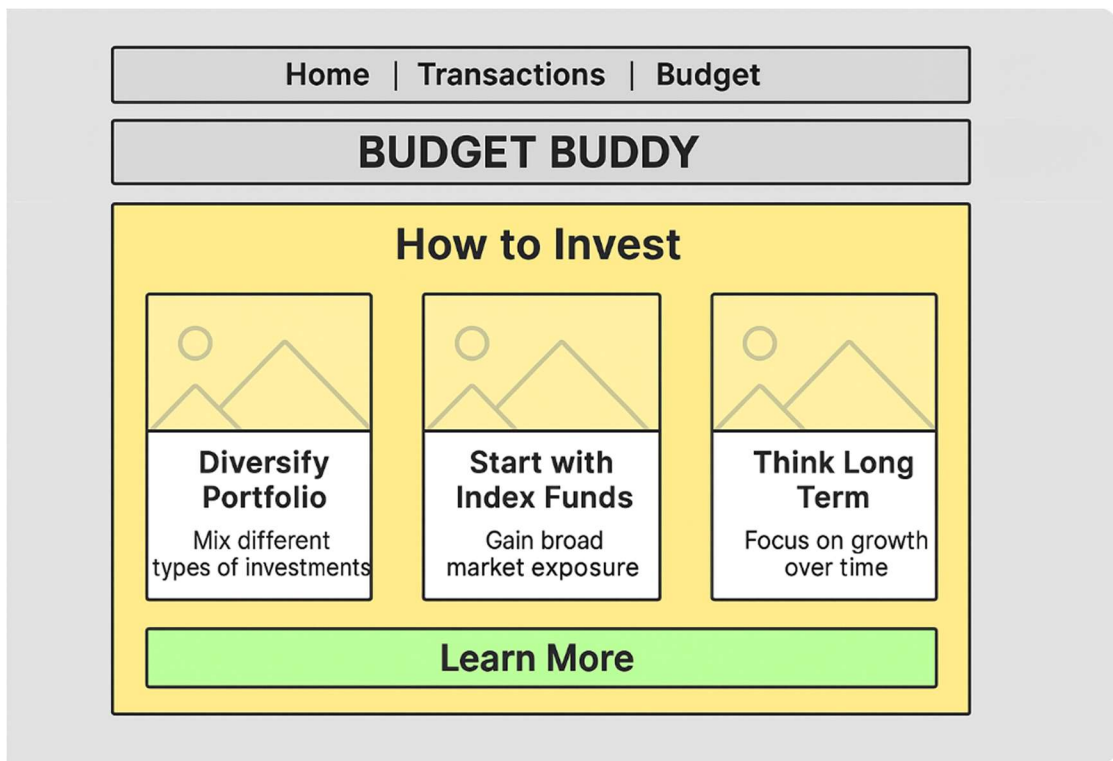


Avoid Impulse Purchases
Wait 24 hours before non-essential spending

Show Challenge

Today's Savings Challenge

- **Investment Page:** Expandable sections for different investment types (e.g., stocks, bonds, ETFs).



- **FAQ Page:** Bootstrap accordion listing common budgeting questions with expandable answers.
- **Authors Page:** Displays team member information via authors.json, ensuring scalability for future contributors.

11. Final Comments

Through this project, we aim to strengthen our understanding of modular JavaScript design, JSON-driven UI architecture, and collaborative Git-based workflows.

We expect to enhance our skills in accessibility, structured data management, and performance optimization.

We are aware of potential risks such as inconsistent JSON formatting, caching issues in static hosting, and browser compatibility.

To mitigate these, we will use the cache: 'no-store' setting in fetch calls, implement validation checks, and test across multiple browsers.